Dali: Lazy Compilation & Kernel Fusion in Dynamic Computation Graphs

Jonathan Raiman
OpenAI
San Francisco, CA
raiman@openai.com

Abstract

Computation graphs underpin many of the large machine learning projects of our time, from computer vision, to machine translation, speech synthesis, or facial recognition. Graph optimizations offer the ability to make these applications run more efficiently, adapt to a variety of hardware platforms, and tighten the iteration cycle from prototype to production. However, generating these graph optimizations requires extensive domain knowledge about hardware and machine learning, and applying these transformations is generally considered too costly to run online.

Due to these difficulties, graph optimizations have only seen applications in static graph frameworks (e.g. TensorFlow, CNTK, Theano, MXNet), while the flexibility and ability to use native control flow has prevented dynamic graph frameworks (e.g. PyTorch, DyNet, Chainer) from benefiting.

We seek to address these challenges by operating on lazily compiled graphs instead. This approach retains the imperative style of dynamic graphs, while also enabling the ability to apply graph optimizations at each evaluation point. We achieve this with Dali through two key components: 1) we amortize the optimization cost by caching our graph optimization passes into reusable Transformation Graphs, 2) we generate fused CUDA kernels using an A* search over a basic cost model of the GPU's memory and parallelism and an admissible heuristic allowing for rapid exploration of the code landscape. We offer preliminary results on Dali while training an image classification task: 1) we find that cached transformations are inexpensive and can automatically replace operations by more efficient primitives or dynamically generated CUDA kernels on the fly, 2) fusing operations reduces memory usage by 45% and runtime by 9%, 3) we observe a $1.79\times$ speedup over TensorFlow.

1 Introduction

Computation frameworks have greatly expanded the ability to transform research ideas into working prototypes and deploy models in production. Perhaps the most important aspect of a shared computation graph language is the ability to apply graph optimization incorporating domain knowledge from a variety of experts and sources to bear on any problem expressed within this representation. Graph optimizations have proven extremely valuable in a variety of cases by providing memory or speed optimizations [1, 2, 3, 4], or automatic code generation using a JIT compiler and an autotuner (e.g. Tensor Comprehensions, XLA, PyTorch Tracing) [5, 6, 7]. There exists rare cases of on-the-fly optimizations [8], however the vast majority of these graph optimizations appear to be too costly to run online, thereby limiting their applicability to static graphs or during conversion to production.

In this paper we introduce Dali, a lazy compiler for dynamic computation graphs, which enables efficient application of graph optimizations while retaining the imperative style and expressivity of

Preprint. Work in progress.

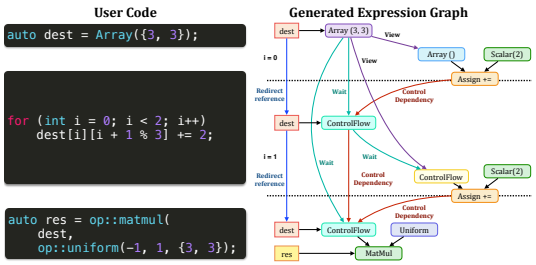

Figure 1: We record all operations in the graph symbolically, by dynamically building up an Expression Graph. Operations that modify memory replace the affected object by the associated transaction operation (assignment, read, etc.) last applied to it, ensuring the user imperative order translates to the same partial order in the computation graph. Dashed line indicates variable `dest` redirected to reflect a memory transaction.

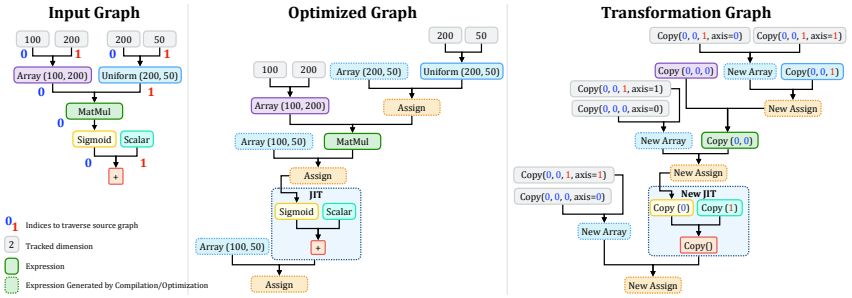

Figure 2: Transformation of an Input Graph into an Optimized Graph, from which we derive a Transformation Graph we cache and can reapply later.

dynamic approaches, depicted in Figure 1. Our solution takes advantage of the observation that most machine learning applications rely on a small set of computation graphs that are evaluated $10^6 - 10^8$ times over the course of training: we compute a graph optimization once, and cache a reusable transformation that is cheaply applied on the next occurrence. Concretely our key contributions are the following:

1. A lazy compiler that integrates a variety of optimization passes in an extensible optimization registry (operation conversion to CuDNN [9], sub-expression elimination, device placement, JIT kernel generation and fusion across reductions, etc.). We overcome the cost of graph optimization by caching our optimization using a method we name Transformation Graphs.
2. We provide preliminary results that our cached graph optimization approach reduces by a factor of $257\times$ the transformation cost, suggesting graph optimizations are applicable to dynamic graphs.
3. We provide a method for generating fused JIT kernels across multiple operations while only exploring a very limited number of solutions. We rely on an A* algorithm [10] that integrates an admissible heuristic for estimating parallelism and constraints that enforce data-consistency in the kernel. We find that fusing multiple operations into a single CUDA kernel leads to a 9% speedup and a 45% memory reduction.

The rest of this paper is structured as follows. In Section 2 we introduce Transformation Graphs, in Section 3 we present our fused CUDA kernel search, in Section 4 we present numerical results investigating overhead, memory and runtime of optimizations. Conclusions are given in Section 5.

2 Transformation Graph

Application and testing of many optimizations against a graph can be costly, and typically grows as $O(N \cdot k)$, where N is the number of nodes and k the number of optimizations.

Our solution relies on caching the transformations applied to the input graph, and reapplying them to graphs that has the same structure and operations, but different contents. To achieve this we hash[1] the input graph P, then iteratively optimize P into an output graph Q.

To recover the transformation that takes P to Q we must trace the effect of each transformation in the optimization passes. Because tracing through passes can be complex or error prone, we instead construct a placeholder target graph $Q_{\text{placeholder}}$ which has the same graph structure as Q, but with all nodes now containing reconstruction information. Each node of $Q_{\text{placeholder}}$ contains one of two types of reconstruction information:

1. Retrieval from the input graph P: if a node in Q was left unchanged from its state in P, then we store a path from the root of the input graph P to the nodes[2]. Marked "Copy" in Fig. 2.
2. A function that constructs the node anew along with some recovery information[3].

To reapply our transformation to a new input graph P' with the same hash as P, we iterate through each node in $Q_{\text{placeholder}}$, using P' as the input graph, and the reconstruction information stored when transforming $P \rightarrow Q$. As a byproduct of optimizing using a cached Transformation Graph, we removed the dependence on the number of optimization passes k in the computational complexity of any future application of the same optimization: we now take time $O(M)$, where M is the number of nodes in the optimized output graph Q.

3 A* Fused CUDA Kernel Search

Automatic CUDA Kernel code generation is an active area of research, with many recent approaches relying on evolutionary or random search techniques to optimize the code in a kernel in a black-box fashion until they arrive at a task-specific efficient implementation [3, 5]. Autotuning approaches have shown impressive performance, but the hour-long search process makes them impractical for online optimization/dynamic computation graphs. We propose to use a search based strategy that relies on a basic cost model of the GPU's memory and parallelism, which does not require compilation to estimate performance, but instead uses a hand designed heuristic that aims to maximize parallelism[4].

To construct the JIT subgraph, we consider each JIT-able Expression which provides information about the for loops and access pattern over its inputs. This information lets us find a safe way of combining multiple reductions and elementwise operations into a single kernel while remaining data-consistent[5]. Loops are annotated with the available ways to parallelize them: Blocks, Threads, Blocks and Threads, or None (e.g. run sequentially).

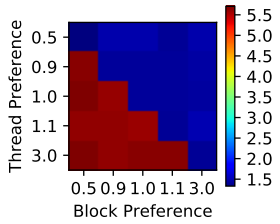

Figure 3: Epoch time as thread vs block usage preference changes.

Framework	Time (s) / Epoch ($\mu \pm \sigma$)	Allocated MB / Step
Dali w/o JIT Fusion	1.44 ± 0.0035	513.33
Dali	1.32 ± 0.0047	343.89
Dali w/o temp	1.37 ± 0.0069	343.86
TensorFlow	2.39 ± 0.0166	1178.00

Table 1: Runtime & Memory Usage

[1]Hashed in bottom-up order, taking care to only include in our hash components that affect the nature of the computation, not its exact contents (e.g. hash the operation type, dimensionality, but not the memory contents or exact dimensions).

[2]Each expression contains zero or more arguments, so a path can be generated by storing a list of indices indicating which argument of a node we must traverse to arrive at the node we want to retrieve.

[3]E.g. if temporary storage is created during our optimization pass to store intermediary results, then we save instructions stating "New Array", with a shape given by retrieving the proper strides and dimensions from the input graph P.

[4]Our heuristic weighs the total number of loops against the number of unique parallelism types exposed, while penalizing the number of loops that have to be run sequentially. Our admissible heuristic sets all unassigned loops to be maximally parallel, thereby providing a valid upper bound on the remaining parallelism opportunities.

[5]Iterators belonging to a loop can be annotated to state they will access data non sequentially, indicating a possible violation of data-consistency if the read data was generated in a different CUDA block.

Table 2: Optimization Time

Step	Time/Call ($\mu \pm \sigma$)	# Calls/10 Epochs	Percent Total
Optimization	41.11ms $\pm$ 320μs	3	0.64%
Apply Cached Transform	106.31 μs $\pm$ 1.00μs	4217	1.17%
Expression hash	53.36 μs $\pm$ 0.160μs	4220	2.33%
Computation	4.366ms $\pm$ 14.29μs	4220	95.85%

Once we have collected all loop variables and data-consistency constraints of the problem, we use A* to find the lowest cost kernel [10]. Empirically, over 80% of the solutions to generating Softmax or LayerNorm[11] are data inconsistent, and our heuristic only evaluates the cost of 13-14% of the total solution space, thereby eliminating many unwanted candidates. We also compared the effect of weighing different aspects of our heuristic and report our results in Figure 3. Specifically we simulate whether threads will parallelize better than blocks, thereby deciding to allocate more variables to use threads instead, and vice-versa. The fastest kernels are found by preferring blocks over threads, while preferring threads to blocks increases runtime $4.23\times$.

4 Results

To investigate whether dynamic compilation is competitive with current approaches, we use a C++ implementation of Dali with a lazy dynamic computation graph, Transformation Graphs, and A* based fused JIT kernel generation. We summarize our timing experiments regarding graph transformation on an image classification Convolutional Neural Network.

We measure memory usage and runtime per epoch over 100 epochs of training on the last 54,000 images of MNIST (28x28 grayscale) using a batch size of 256, on a 12-core 3.60GHz Intel i7-6850K CPU with an NVIDIA Titan X Pascal. We compare four configurations: Dali with or without JIT Fusion, Dali with no temporary storage in the generated code, and TensorFlow 1.11. We report our results in Table 1, and observe a $1.79\times$ speedup of Dali over TensorFlow [12] while the same CUDA 10, CuDNN 7.3.1 primitives [9] in both frameworks (Table 1).

We also time finer grain steps of the graph optimization in Table 2. In those measurements, we note that hashing and transformation is $\approx 257\times$ faster than the initial optimization pass. Only 3 distinct computation graphs were used over the course of training[6]. These preliminary results suggest that lazy compilation over dynamic graphs can be time efficient given the amortization and repeated use of computation graphs.

5 Conclusion

We have introduced Dali, a lazy compiler for dynamic computation graphs. We have shown how we can amortize the cost of graph optimization by recognizing repeated occurences, reducing transformation time by a factor of $\approx 257\times$, while also removing the number of optimization passes from the computational complexity of the graph transformation. We have also presented a method for generating fused JIT kernels using search by using a cost model that respects data-consistency, maximizes parallelism. We believe that broadening the applicability of graph optimizations can have a strong impact on a variety of machine learning applications by allowing easy integration of systems domain knowledge without requiring expertise from the end user.

As future work we intend to further investigate the performance limitations of the technique on more complex models and tasks. We believe that the graph transformation work presented here could benefit from a smarter search and application strategy, for instance by incorporating data-driven heuristics into the search process discovered either through tuning or by exploiting the value function derived from reinforcement learning the optimization policy following [13, 14], while we also recognize that our search heuristic can also be complimentary to data-driven approaches by dramatically pruning the search space.

[6] 1) Initialization of the weights, 2) Forward pass only, 3) Forward and backward passes.

References

[1] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

[2] Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Summer Deng, Roman Dzhabarov, James Hegeman, Roman Levenstein, Bert Maher, Satish Nadathur, Jakob Olesen, et al. Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*, 2018.

[3] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Tvm: end-to-end compilation stack for deep learning. In *SysML Conference*, 2018.

[4] Salimans, Tim and Bulatov, Yaroslav. Saving memory using Gradient Checkpointing. `https://github.com/openai/gradient-checkpointing`, 2018. [Online; accessed 19-October-2018].

[5] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions. *arXiv preprint arXiv:1802.04730*, 2018.

[6] Tensorflow. XLA Overview. `https://www.tensorflow.org/performance/xla/`, 2018. [Online; accessed 19-October-2018].

[7] Torch Contributors. Torch Script. `https://pytorch.org/docs/master/jit.html`, 2018. [Online; accessed 19-October-2018].

[8] Graham Neubig, Yoav Goldberg, and Chris Dyer. On-the-fly operation batching in dynamic computation graphs. In *Advances in Neural Information Processing Systems*, pages 3971–3981, 2017.

[9] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cudnn: Efficient primitives for deep learning. *arXiv preprint arXiv:1410.0759*, 2014.

[10] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[11] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *stat*, 1050:21, 2016.

[12] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: a system for large-scale machine learning. In *OSDI*, volume 16, pages 265–283, 2016.

[13] Azalia Mirhoseini, Anna Goldie, Hieu Pham, Benoit Steiner, Quoc V Le, and Jeff Dean. A hierarchical model for device placement. 2018.

[14] Azalia Mirhoseini, Hieu Pham, Quoc Le, Mohammad Norouzi, Samy Bengio, Benoit Steiner, Yuefeng Zhou, Naveen Kumar, Rasmus Larsen, and Jeff Dean. Device placement optimization with reinforcement learning. 2017.